



Easy, fast, and cheap LLM serving for everyone

| [Documentation](#) | [Blog](#) | [Paper](#) | [Twitter/X](#) | [User Forum](#) | [Developer Slack](#) |

🔥 We have built a vLLM website to help you get started with vLLM. Please visit vllm.ai to learn more. For events, please visit vllm.ai/events to join us.

YOCO Fast 优化 (2026-09-05)

本分支 dev/yoco-fast-optimization-20260905 为 YOCO L3 的 --fast 增加了按 token 行数选择 MoE backend 的策略，主要改善大批量 Prefill。需使用本分支的 vLLM；下方上游安装说明中的发行版不包含这项开发改动。

启用条件与实现

在 B200 / SM100、L3 BF16、无量化、TP/DP/PP/CP 均为 1、standalone 且启用 --kv-sharing-fast-prefill 时，--fast 默认启用新策略。还要求 FlashInfer CUTLASS 可用、未显式启用 FlashInfer autotune，且 scheduler 的 token budget 足够覆盖切换阈值。已验证 FlashInfer 版本为 0.6.8.post1。

```
threshold = max(1024, max_num_seqs + 1, max_cudagraph_capture_size + 1)
M >= threshold : FlashInfer CUTLASS heuristic
M < threshold : 原 Fast Triton 调参与 MoE sum
```

M 是此次 MoE 调用的 token 行数，不是请求数。测试配置中的阈值为 1024；阈值随最大请求数和 graph bucket 提高，以保留已配置的纯 Decode 图走 Triton。两条路径处理相同的 W13 布局与 SwiGLU clamp，并通过跨层共享 workspace 控制临时显存。

如需关闭新策略，在现有启动命令中添加：

```
--additional-config '{"yoco_fast_standalone_flashinfer_moe": false}'
```

实测结果

以下比较均使用同一张物理 B200、同一 L3 checkpoint 与 BF16。短测按 A/B/B/A 顺序，每档每版共 8 次，报告中位数；Decode 工作负载吞吐包含 Prefill 和调度时间。

指标	优化前 → 优化后 / 变化
固定形状 Prefill 吞吐	提高 5.3%-8.9%
Decode 工作负载吞吐	-0.4% 到 +1.4%，基本持平；单独 TPOT 未显示明确加速
AIPerf 输出吞吐	1029.27 → 1031.97 tok/s, +0.26% ，基本持平
AIPerf TTFT P95	2.949 → 2.834 s，降低 3.92%
AIPerf E2E P95	66.285 → 63.022 s，降低 4.92%
AIPerf ITL P95	306.057 → 311.707 ms，增加 1.85%

长测复用公开 Mooncake FAST’25 toolagent_trace 的 300-900 秒窗口：600 秒固定到达、1×、3643 请求、context 上限 81920、每轮独立 cache salt，并等待全部请求排空。主比较两端的实际输入/输出 token 数逐请求一致。

40 项配置与 kernel 测试通过。候选首次长测出现 1 次 HTTP 连接重置，完整性门禁未通过；保持参数不变后完整补跑，**3643/3643** 请求通过客户端和服务端审计。共享节点、单次基线与候选补跑及不同预热历史使这组长测属于诊断结果，不代表稳定容量；失败记录均保留在报告中。

Fast 不保证 **bitwise**：新旧版本在部分高 batch 生成序列上有差异。BF16、128 experts、Top-8 和 clamp 保留，本轮没有修改 Align 前向 kernel，也未完成模型质量评估。

详细资料位于本地工作区同级的 yoco_results/ 目录：

- [Fast 专项报告：实现、参数、验证与失败记录](#)
- [综合开发报告：Align、训练反向与 Fast 优化](#)
- [短测数据与长测数据](#)

About

vLLM is a fast and easy-to-use library for LLM inference and serving.

Originally developed in the [Sky Computing Lab](#) at UC Berkeley, vLLM has grown into one of the most active open-source AI projects built and maintained by a diverse community of many dozens of academic institutions and companies from over 2000 contributors.

vLLM is fast with:

- State-of-the-art serving throughput
- Efficient management of attention key and value memory with **PagedAttention**
- Continuous batching of incoming requests, chunked prefill, prefix caching
- Fast and flexible model execution with piecewise and full CUDA/HIP graphs
- Quantization: FP8, MXFP8/MXFP4, NVFP4, INT8, INT4, GPTQ/AWQ, GGUF, compressed-tensors, ModelOpt, TorchAO, and [more](#)

- Optimized attention kernels including FlashAttention, FlashInfer, TRTLLM-GEN, FlashMLA, and Triton
- Optimized GEMM/MoE kernels for various precisions using CUTLASS, TRTLLM-GEN, CuTeDSL
- Speculative decoding including n-gram, suffix, EAGLE, DFlash
- Automatic kernel generation and graph-level transformations using torch.compile
- Disaggregated prefill, decode, and encode

vLLM is flexible and easy to use with:

- Seamless integration with popular Hugging Face models
- High-throughput serving with various decoding algorithms, including *parallel sampling*, *beam search*, and more
- Tensor, pipeline, data, expert, and context parallelism for distributed inference
- Streaming outputs
- Generation of structured outputs using xgrammar or guidance
- Tool calling and reasoning parsers
- OpenAI-compatible API server, plus Anthropic Messages API and gRPC support
- Efficient multi-LoRA support for dense and MoE layers
- Support for NVIDIA GPUs, AMD GPUs, and x86/ARM/PowerPC CPUs. Additionally, diverse hardware plugins such as Google TPUs, Intel Gaudi, IBM Spyre, Huawei Ascend, Rebellions NPU, Apple Silicon, MetaX GPU, and more.

vLLM seamlessly supports 200+ model architectures on Hugging Face, including:

- Decoder-only LLMs (e.g., Llama, Qwen, Gemma)
- Mixture-of-Expert LLMs (e.g., Mixtral, DeepSeek-V3, Qwen-MoE, GPT-OSS)
- Hybrid attention and state-space models (e.g., Mamba, Qwen3.5)
- Multi-modal models (e.g., LLaVA, Qwen-VL, Pixtral)
- Embedding and retrieval models (e.g., E5-Mistral, GTE, ColBERT)
- Reward and classification models (e.g., Qwen-Math)

Find the full list of supported models [here](#).

Getting Started

Install vLLM with `uv` (recommended) or `pip`:

```
uv pip install vllm
```

Or [build from source](#) for development.

Visit our [documentation](#) to learn more.

- [Installation](#)
- [Quickstart](#)

- [List of Supported Models](#)

Contributing

We welcome and value any contributions and collaborations. Please check out [Contributing to vLLM](#) for how to get involved.

Citation

If you use vLLM for your research, please cite our [paper](#):

```
@inproceedings{kwon2023efficient,  
  title={Efficient Memory Management for Large Language Model Serving with PagedAttention},  
  author={Woosuk Kwon and Zhuohan Li and Siyuan Zhuang and Ying Sheng and Lianmin Zheng and  
Cody Hao Yu and Joseph E. Gonzalez and Hao Zhang and Ion Stoica},  
  booktitle={Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles},  
  year={2023}  
}
```

Contact Us

- For technical questions and feature requests, please use GitHub [Issues](#)
- For discussing with fellow users, please use the [vLLM Forum](#)
- For coordinating contributions and development, please use [Slack](#)
- For security disclosures, please use GitHub's [Security Advisories](#) feature
- For collaborations and partnerships, please contact us at collaboration@vllm.ai

Media Kit

- If you wish to use vLLM's logo, please refer to [our media kit repo](#)